Credit Card Fraud Detection

An End-to-End MLOps System

Project Report

Name: Tejasri Suthapalli | Roll Number: NA21B075
Repository: github.com/encylope/MLOPS_Project

1. Executive Summary

This project delivers a production-grade, real-time Credit Card Fraud Detection system built entirely on MLOps principles. The system classifies incoming credit card transactions as fraudulent or legitimate using a machine learning model trained on the Kaggle Credit Card Fraud dataset. Predictions are exposed through a REST API backed by a React web portal.

The implementation covers the complete ML lifecycle: automated data ingestion via Apache Airflow, data and model versioning with DVC and MLflow, containerised deployment with Docker Compose, and real-time monitoring with Prometheus and Grafana. The project was developed and run locally on Windows 11 with Docker Desktop.

Key Results

Metric	Achieved Value
ROC-AUC (test set)	0.9598
AUC-PR (test set)	0.8134
F1 Score — fraud class	0.7947
Precision — fraud class	0.7792
Recall — fraud class	0.8108
Overall accuracy	0.9993
Average inference latency	~60ms (target: <200ms)
Unit tests	9 passed / 9 total
Docker services running	6 / 6
MLflow runs logged	5 training runs
DVC pipeline stages	6 stages (validate → evaluate)

2. Problem Statement

Financial fraud causes billions of dollars in losses for banks and payment platforms annually. Traditional rule-based fraud detection systems suffer from high false positive rates and inability to adapt to evolving fraud patterns. Machine learning offers a data-driven approach that can identify subtle patterns across many features simultaneously.

The specific objective of this system is to build a real-time classification engine that can:

- Accept 30 transaction features (V1–V28, Amount, Time) and return a fraud probability within 200ms
- Provide a calibrated risk tier (LOW / MEDIUM / HIGH) alongside the probability score
- Be reproducible — every prediction can be traced back to a specific model version, training run, and dataset version
- Be observable — all inference metrics are streamed to Prometheus and visualised in Grafana in near real time

2.1 Dataset

Source: Kaggle Credit Card Fraud Detection dataset (mlg-ulb/creditcardfraud). This is a widely-used benchmark dataset for fraud detection research.

Property	Detail
File	creditcard.csv (144 MB)
Total transactions	284,807
Fraudulent transactions	492 (0.1727%)
Legitimate transactions	284,315 (99.83%)
Number of features	30 input features + 1 target
Feature names	V1 through V28 (PCA-transformed), Amount, Time
Target column	Class (0 = legitimate, 1 = fraud)
Time period	Two days of European cardholder transactions

Note: V1 through V28 are the result of PCA transformation applied by the dataset creators to protect cardholder privacy. The original feature names and meanings are not disclosed. Amount and Time are the only non-anonymised features.

3. System Architecture

3.1 Layer Overview

Layer	Components and Tools
Data Engineering	Apache Airflow 2.9 DAG, Python preprocessing scripts, DVC pipeline
Training and Tracking	XGBoost 2.0, scikit-learn 1.5, MLflow 2.13, SMOTE (imbalanced-learn)
Model Serving	FastAPI 0.111, Uvicorn, MLflow Model Registry, Pydantic 2.7
Frontend	React 18, Vite 5, Recharts, served by nginx
Containerisation	Docker, Docker Compose (6 services)
Monitoring	Prometheus 2.51, Grafana 10.4, prometheus-client 0.20
CI/CD	GitHub Actions (4 jobs), DVC, Git

3.2 Design Principles

The following principles guided all design decisions:

- Loose coupling: The frontend and backend are independent Docker containers. They communicate exclusively through REST API calls. No shared code, filesystem, or database.
- Reproducibility: Every experiment is uniquely identified by its Git commit hash (tracked in MLflow tags) and MLflow run ID. Running dvc repro from any commit recreates all pipeline outputs deterministically.
- Automation: All six pipeline stages (validate through evaluate) run unattended via dvc repro. The Airflow DAG automates daily data ingestion.
- Environment parity: The MLproject file defines entry points and a python_env.yaml specifying exact package versions. Docker Compose ensures identical environments across development and deployment.
- Observability: The FastAPI backend exposes five custom Prometheus metrics. Prometheus scrapes them every 10 seconds. Grafana visualises request rates, fraud rates, and latency in near real time.

3.3 Complete Data Flow

The end-to-end data flow through the system is as follows:

1. creditcard.csv is placed in data/raw/ and tracked with DVC (dvc add)
2. Apache Airflow DAG (fraud_data_ingestion_pipeline) runs the ingestion stages
3. DVC pipeline (dvc repro) runs all six stages in dependency order
4. Training script logs all parameters, metrics, and artifacts to MLflow
5. Trained model is registered in MLflow Model Registry as fraud-detector
6. Model is promoted to the production alias in the registry

7. FastAPI backend loads the model from the registry on startup
8. React frontend sends transaction features to POST /api/v1/predict
9. Backend returns fraud probability, risk tier, model version, and latency
10. Prometheus scrapes /metrics every 10 seconds for monitoring

4. Data Engineering Pipeline

4.1 Apache Airflow DAG

The data ingestion pipeline is implemented as an Airflow DAG (fraud_data_ingestion_pipeline) in airflow/dags/fraud_ingestion_dag.py. It uses a SequentialExecutor with a local SQLite metadata database. The DAG is scheduled daily (@daily) but can also be triggered manually.

The DAG contains four PythonOperator tasks in a linear dependency chain:

Task ID	Description and Key Operations
validate_raw_data	Loads creditcard.csv. Checks that all 31 expected columns are present (V1-V28, Amount, Time, Class). Counts null values per column. Verifies row count >= 1000. Computes and logs fraud rate (0.1727%). Saves validation_report.json to data/processed/.
preprocess_data	Removes extreme outliers: rows where Amount > 99.9th percentile (cap = ~25,691 USD). Resulting rows: 284,523. Applies StandardScaler to Amount and Time columns only. Saves fitted scaler parameters (mean, scale) to scaler_params.json. Saves creditcard_processed.csv.
feature_engineering	Computes baseline statistics for all 30 features: mean, std, min, max, 25th percentile, 75th percentile. These baselines are saved to feature_baseline.json and used as reference for future data drift detection. Saves features.csv.
split_dataset	Performs stratified train/validation/test split (70% / 15% / 15%) with random_state=42. Stratification ensures the 0.17% fraud rate is preserved in all splits. Saves train.csv, val.csv, test.csv to data/features/.

4.2 DVC Pipeline

In addition to Airflow, the entire pipeline is defined in dvc.yaml as a DVC pipeline with six stages. This enables fully reproducible reruns using dvc repro.

DVC Stage	Command
validate	python scripts/validate_data.py
preprocess	python scripts/preprocess.py
feature_engineering	python scripts/feature_engineering.py

split	python scripts/split_data.py
train	python scripts/train.py --model-type xgboost
evaluate	python scripts/evaluate.py

Each stage declares its dependencies (input files and scripts) and outputs (produced files). DVC tracks content hashes of all inputs and outputs in dvc.lock. If no inputs have changed, DVC skips the stage. This makes the pipeline efficient for iterative development.

4.3 Dataset Split Results

Split	Total Rows
Training set	199,166 rows — 344 fraud (0.1727%)
Validation set	42,678 rows — 74 fraud (0.1734%)
Test set	42,679 rows — 74 fraud (0.1734%)

4.4 Data Versioning

The raw dataset (creditcard.csv, 144 MB) is tracked by DVC rather than Git. Running dvc add data/raw/creditcard.csv creates a small pointer file (creditcard.csv.dvc) that is committed to Git. The actual CSV is listed in .gitignore. All derived artifacts (processed CSVs, features, metrics) are similarly tracked through dvc.lock.

5. Model Development

5.1 Model Selection

XGBoost (Extreme Gradient Boosting) was selected as the primary model for the following reasons:

- Strong performance on tabular data with class imbalance
- Built-in support for the scale_pos_weight parameter to handle imbalanced classes
- Fast inference time — single prediction in approximately 5ms on local hardware
- Feature importance is natively available for interpretability
- Compatible with MLflow sklearn flavour for seamless model registry integration

5.2 Handling Class Imbalance

The dataset is highly imbalanced: only 0.17% of transactions are fraudulent. Two complementary strategies were used:

SMOTE Oversampling

SMOTE (Synthetic Minority Oversampling Technique) from imbalanced-learn was applied to the training set with a sampling_strategy of 0.10. This generated synthetic fraud samples until the fraud-to-legitimate ratio reached 10%.

Dataset	Rows
Training set before SMOTE	199,166 rows — 344 fraud (0.17%)
Training set after SMOTE	218,704 rows — 19,882 fraud (9.09%)
Synthetic samples generated	19,538 fraud samples

scale_pos_weight

XGBoost's scale_pos_weight parameter was set to 10, further biasing the model toward correctly classifying the minority fraud class during training.

5.3 Model Hyperparameters

Hyperparameter	Value
n_estimators	200
max_depth	6
learning_rate	0.05
scale_pos_weight	10
eval_metric	aucpr
SMOTE sampling_strategy	0.10
SMOTE random_state	42
Train/val/test random_state	42

5.4 Decision Threshold Tuning

Rather than using the default threshold of 0.5 for binary classification, the decision threshold was tuned on the validation set. The threshold was swept from 0.1 to 0.9 in steps of 0.05, and the value maximising the F1 score on the fraud class was selected.

Optimal threshold found: 0.85 (validation F1 = 0.7564). This threshold is logged as a parameter in MLflow and applied at inference time.

5.5 Feature Column Order

An important implementation detail: the XGBoost model expects features in the exact order they were provided during training. The training pipeline uses the following column order:

```
FEATURE_COLUMNS = ['Time', 'V1', 'V2', ..., 'V28', 'Amount']
```

The backend model service (model_service.py) uses this same order when constructing the feature DataFrame from incoming API requests. A mismatch between training order and inference order causes a ValueError from XGBoost, which was identified and resolved during development.

6. MLflow Experiment Tracking

6.1 Experiment Setup

MLflow is used for experiment tracking, model versioning, and model serving. The tracking server runs as a Docker container (fraud_mlflow) at http://localhost:5050, backed by a SQLite database (mlflow/mlflow.db) and a local artifact store (mlflow/artifacts/).

Experiment name: credit-card-fraud-detection

6.2 What is Logged Per Run

Category	Logged Items
Parameters	model_type, n_estimators, max_depth, learning_rate, scale_pos_weight, smote_sampling_strategy, smote_random_state, decision_threshold
Metrics	test_roc_auc, test_auc_pr, test_f1_fraud, test_precision_fraud, test_recall_fraud, test_accuracy, avg_inference_latency_ms, training_time_seconds, train_rows_after_smote
Artifacts	confusion_matrix.png, roc_curve.png, feature_importance.png, classification_report.json, metrics/train_metrics.json
Tags	git_commit (short hash), dvc_hash, dataset name, model_type, engineer username
Model	Registered as fraud-detector in MLflow Model Registry

6.3 Model Registry and Versioning

Five versions of the fraud-detector model were registered during development:

Version	Description
v1	First training run (local MLflow at port 5000)
v2	Second run — metrics saved to train_metrics.json fixed
v3	Third run — evaluate.py fixed with roc_curve.csv output

v4	Trained inside Docker container (fixed Windows artifact path issue)
v5	Additional Docker training run

Version 4 is the production model. It is identified by the alias production in the MLflow Model Registry. The FastAPI backend loads the model using models:/fraud-detector@production and resolves the artifact path to the Docker-internal /mlflow/artifacts/ directory.

6.4 Artifact Path Resolution

A key engineering challenge encountered: MLflow stores the artifact source path at the time of model registration. When training is run on Windows locally, MLflow saves a Windows path (e.g., file:C:/Users/Tejasri/...). When the Docker container then attempts to load the model, it cannot resolve the Windows path.

Solution: Training was re-run inside the fraud_backend Docker container using docker exec. This caused MLflow to record a Linux path (/mlflow/artifacts/...) which the container can resolve correctly. Additionally, the model_service.py includes a path-normalisation step using regex to convert any remaining Windows paths to Linux paths as a fallback.

7. Deployment

7.1 Docker Services

The entire system runs as six Docker containers orchestrated by Docker Compose (docker-compose.yml). All containers share the fraud_net bridge network.

Container	Image / Port	Purpose
fraud_backend	Custom (python:3.11-slim) / 8800	FastAPI inference engine
fraud_frontend	Custom (node:20-alpine + nginx) / 3030	React web portal
fraud_mlflow	ghcr.io/mlflow/mlflow:v2.13.0 / 5050	Experiment tracking and model registry
fraud_airflow	apache/airflow:2.9.0-python3.11 / 8082	Pipeline orchestration
fraud_prometheus	prom/prometheus:v2.51.0 / 9095	Metrics scraping
fraud_grafana	grafana/grafana:10.4.0 / 3035	Dashboard visualisation

Note: Non-standard ports (8800 instead of 8000, 5050 instead of 5000, etc.) were used because this machine runs multiple previous assignment containers that occupy the standard ports. The fraud project containers use ports in the 3030, 5050, 8082, 8800, 9095, and 3035 range.

7.2 Backend: FastAPI

The backend is a FastAPI application (backend/app/main.py) running with two Uvicorn workers. It exposes the following endpoints:

Endpoint	Description
GET /health	Liveness check. Always returns 200 with model_loaded status.
GET /ready	Readiness check. Returns 503 if the model has not been loaded yet.
POST /api/v1/predict	Single-transaction fraud classification. Accepts all 30 features, returns fraud_probability, risk_level (LOW/MEDIUM/HIGH), model_version, inference_time_ms, transaction_id (UUID4).
POST /api/v1/predict/batch	Batch prediction for up to 100 transactions. Returns all predictions plus fraud_count and processing_time_ms.
POST /api/v1/reload-model	Hot-reloads the production model from MLflow registry without restarting the container.
GET /api/v1/pipeline/status	Returns DVC pipeline stage statuses as JSON.
GET /api/v1/pipeline/dag	Returns the DVC DAG structure as text.
GET /metrics	Prometheus metrics endpoint in text exposition format.

7.3 Frontend: React Portal

The frontend is a React 18 single-page application built with Vite 5 and served by nginx. It has three pages:

Page	Description
Predict	Main fraud assessment page. Contains 30 input fields (V1–V28, Amount, Time). Load legitimate and Load fraud buttons populate the form with sample transactions. The Analyze Transaction button calls POST /api/v1/predict and displays the result card with probability bar, risk level badge, model version, and latency.
Pipeline	Shows the six DVC pipeline stages visually. Displays current model status (loaded, version) from /health. Links to all six external monitoring tools (Airflow, MLflow, Prometheus, Grafana).
Dashboard	Live metrics charts using Recharts. Shows total requests, fraud detections, fraud rate, and average latency. Charts refresh every 5 seconds. Links to external tools for full dashboards.

7.4 MLproject

The MLproject file defines project entry points for mlflow run commands, enabling standardised execution of pipeline stages. The python_env.yaml specifies exact package versions (mlflow==2.13.0, scikit-learn==1.5.0, xgboost==2.0.3, etc.) to ensure environment parity.

8. Monitoring and Observability

8.1 Prometheus Instrumentation

Five custom Prometheus metrics are instrumented in the FastAPI backend using the prometheus-client library. The /metrics endpoint is mounted as an ASGI sub-application and scraped by Prometheus every 10 seconds.

Metric	Type and Description
fraud_api_requests_total	Counter. Labelled by method, endpoint, and HTTP status code. Tracks total API call volume.
fraud_api_request_duration_seconds	Histogram. Labelled by endpoint. Records request latency with buckets at 0.005s, 0.01s, 0.025s, 0.05s, 0.1s, 0.25s, 0.5s, 1s, 2.5s.
fraud_predictions_total	Counter. Labelled by prediction outcome (fraud or legitimate). Tracks model output distribution.
fraud_model_confidence_last	Gauge. Stores the confidence score of the most recent prediction. Useful for detecting model degradation.
fraud_active_model_version	Gauge. Stores the currently loaded model version number from the MLflow registry.

An HTTP middleware is registered on the FastAPI app to automatically increment fraud_api_requests_total and record fraud_api_request_duration_seconds for every request, regardless of endpoint.

8.2 Prometheus Configuration

The Prometheus configuration (monitoring/prometheus/prometheus.yml) defines two scrape jobs:

- fraud-backend: scrapes http://backend:8000/metrics every 10 seconds
- prometheus: scrapes its own metrics at http://localhost:9090/metrics

The Prometheus targets page at http://localhost:9095/targets confirms both targets are UP with scrape durations of approximately 12–14ms.

8.3 Grafana

Grafana is provisioned with the Prometheus datasource automatically via monitoring/grafana/datasources.yml. The Fraud Detection dashboard folder is provisioned on container startup via monitoring/grafana/dashboards/dashboard.yml.

A live dashboard has been created manually showing the fraud_api_requests_total metric over time, displaying the accumulated API call history including the error recovery period during model loading, and the successful prediction calls once the model was loaded correctly.

9. Testing

9.1 Unit Tests

Unit tests are located in backend/tests/test_api.py and run with pytest. The tests use FastAPI's TestClient and unittest.mock to isolate the API layer from the MLflow dependency.

Test ID	Description and Result
TC-001	PASSED — GET /health returns HTTP 200 with status=healthy and model_loaded field present
TC-002	PASSED — GET /ready returns HTTP 503 with status=not_ready when model is not loaded (mocked)
TC-003	PASSED — Valid 30-field TransactionRequest passes Pydantic validation without error
TC-004	PASSED — TransactionRequest with Amount=-10.0 raises Pydantic ValidationError (ge=0.0 constraint)
TC-005	PASSED — _compute_risk_level(0.1) returns LOW, _compute_risk_level(0.29) returns LOW
TC-006	PASSED — _compute_risk_level(0.7) returns HIGH, _compute_risk_level(0.99) returns HIGH
TC-007	PASSED — POST /api/v1/predict/batch with empty transactions list returns HTTP 422
TC-008	PASSED — POST /api/v1/predict/batch with 101 transactions returns HTTP 422
TC-009	PASSED — PredictionResponse model instantiates correctly with valid field values

Summary: 9 test cases executed. 9 passed. 0 failed. Test run time: approximately 13 seconds.

9.2 CI/CD Test Execution

Tests are automatically executed on every push to main via the GitHub Actions workflow (.github/workflows/ci.yml). The Lint and Unit Tests job installs all backend dependencies, creates required directories, and runs pytest with coverage reporting.

9.3 Acceptance Criteria

Criterion	Acceptance Value and Outcome
AC-01: Fraud class F1 score	Target >= 0.85 — Achieved: 0.7947 (close, slight shortfall due to threshold = 0.85)
AC-02: ROC-AUC	Target >= 0.95 — Achieved: 0.9598 (exceeds target)
AC-03: Inference latency	Target < 200ms — Achieved: ~60ms (exceeds target significantly)
AC-04: Unit tests pass	100% — Achieved: 9/9
AC-05: All Docker services healthy	6/6 — Achieved

Note on AC-01: The F1 score of 0.7947 is slightly below the 0.85 target. This is attributable to the conservative decision threshold of 0.85, which was chosen on the validation set to maximise F1 on that split. The ROC-AUC of 0.9598 confirms the model's strong discriminative ability. Lowering the threshold to 0.5 yields higher recall at the cost of precision.

10. Source Control and CI/CD

10.1 Git Commit History

Commit	Description
66286dd	Initial commit (blank README from GitHub)
cca23d7	feat: initial project scaffold with MLOps structure
d649173	feat: complete ML pipeline — training + evaluation done
e0329b1	feat: complete MLOps project — all services running
Latest	fix: CI pipeline — fix test env vars and directory setup

10.2 GitHub Actions CI/CD Pipeline

Four jobs run on every push to main:

Job	Steps and Status
Lint and Unit Tests	Installs backend dependencies, creates mlflow/ and metrics/ directories, runs ruff linting (continue-on-error), runs pytest with coverage. Status: Passing.
DVC Pipeline Status	Installs DVC 3.51.2 with pinned pathspec version, initialises DVC, runs dvc dag to validate pipeline definition. Status: Passing.
Build Docker Images	Builds fraud-backend and fraud-frontend Docker images using their respective Dockerfiles, validates docker compose config. Status: Passing.

Integration Smoke Test	Starts MLflow and backend containers, waits for startup, calls GET /health on port 8800, tears down. Status: Passing.

10.3 DVC Data and Model Versioning

DVC tracks the following files and directories:

- data/raw/creditcard.csv — raw dataset (144 MB, stored by DVC, not in Git)
- data/processed/ — validation report, scaler params, processed CSV
- data/features/ — feature CSV, baseline JSON, train/val/test splits
- metrics/train_metrics.json — training metrics for DVC diff
- metrics/roc_curve.csv — ROC curve data for DVC plots

The dvc.lock file records the MD5 hash of every stage input and output, ensuring that dvc repro can detect exactly which stages need re-execution after any change.

11. Technology Stack

Category	Technology and Version
API Framework	FastAPI 0.111.0 + Uvicorn 0.30.0
ML Model	XGBoost 2.0.3
Preprocessing	scikit-learn 1.5.0 (StandardScaler, train_test_split)
Class Imbalance	imbalanced-learn 0.12.3 (SMOTE)
Data Processing	pandas 2.2.2, numpy 1.26.4
Experiment Tracking	MLflow 2.13.0
Data Versioning	DVC 3.67.1
Pipeline Orchestration	Apache Airflow 2.9.0 (SequentialExecutor)
Schema Validation	Pydantic 2.7.1
Containerisation	Docker + Docker Compose v5.0.1
Monitoring (metrics)	prometheus-client 0.20.0
Monitoring (scraping)	Prometheus 2.51.0
Monitoring (visualisation)	Grafana 10.4.0
Frontend Framework	React 18.3.1 + Vite 5.2.13
Frontend Charts	Recharts 2.12.7
Frontend Serving	nginx (alpine)
HTTP Client	axios 1.7.2
CI/CD	GitHub Actions
Python Version	3.11 (Docker), 3.12.10 (local Windows)

Operating System	Windows 11 (development), Linux (Docker containers)

12. Known Issues and Limitations

Issue	Description and Status
F1 score slightly below target	Achieved 0.7947 vs target 0.85. The model's discriminative ability (ROC-AUC 0.9598) is strong. The F1 gap is due to the conservative 0.85 threshold. Can be improved by lowering the threshold or by training with additional hyperparameter tuning.
Airflow uses SQLite	The Airflow instance uses a SQLite metadata database with SequentialExecutor. This means only one task runs at a time and the database is not suitable for concurrent writes. In production, PostgreSQL with CeleryExecutor would be used.
MLflow uses SQLite	The MLflow tracking server uses a SQLite backend store. For concurrent multi-user access, a PostgreSQL backend would be more appropriate.
Windows artifact path issue	MLflow registered artifact paths as Windows file paths when training ran locally. This required re-running training inside the Docker container and adding path normalisation logic to the model service. This is documented in Section 6.4.
Port conflicts	The development machine runs multiple assignment containers, occupying standard ports (3000, 5000, 8000, 8080, 9090). Non-standard ports were used for this project. This is noted in Section 7.1.
DVC remote storage	DVC is initialised but no remote storage is configured. Data is only available locally. A DVC remote (e.g., S3, GCS, or a local NFS mount) would be needed for team collaboration.

13. Conclusion

This project successfully implements a complete, production-grade MLOps system for real-time credit card fraud detection. The system achieves a ROC-AUC of 0.9598 and an average inference latency of approximately 60ms, well within the 200ms requirement.

All six Docker services run reliably. All nine unit tests pass. The complete pipeline from raw data ingestion through model deployment is automated via DVC and Airflow. Experiment tracking, model versioning, and model serving are managed through MLflow. Prometheus scrapes backend metrics every 10 seconds and Grafana visualises them in near real time.

The project encountered and resolved several real-world engineering challenges: class imbalance handling with SMOTE, decision threshold tuning on validation data, Docker artifact path resolution, port conflict management, and model hot-reloading without container restarts. These challenges and their solutions are documented throughout this report.

The codebase is version-controlled on GitHub (encylope/MLOPS_Project) with a CI/CD pipeline that automatically runs linting, unit tests, Docker image builds, and an integration smoke test on every push to main.

End of Report